you ask for a burrito, it can put all the parts together and hand you your meal!

NOTE Under the hood, the built-in ASP.NET Core DI container uses optimized reflection to create dependencies, but different DI containers may use other approaches. The `Add*` APIs are the only way to register dependencies with the built-in container; there's no support for using external configuration files to configure the container, for example.

The service type and implementation type are the same for `NetworkClient` and `MessageFactory`, so there's no need to specify the same type twice in the `AddScoped` method—hence, the slightly simpler signature.

NOTE The `EmailSender` instance is registered only as an `IEmailSender`, so you can't resolve it by requesting the specific `EmailSender` implementation; you must use the `IEmailSender` interface.

These generic methods aren't the only ways to register services with the container. You can also provide objects directly or by using lambdas, as you'll see in section 9.2.

9.2 Registering services using objects and lambdas

As I mentioned in section 9.1, I didn't quite register all the services required by `EmailSender`. In the previous examples, `NetworkClient`